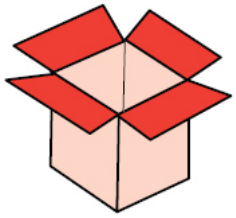
2

Storing values

This chapter demonstrates the various PHP containers in which data can be stored.

Creating variables



A “variable” is a named container in a PHP script in which a data value can be stored. The stored value can be referenced using the variable’s name and changed (varied) as the script proceeds. The script author can choose any name for a variable providing it adheres to these three naming conventions:

- Names must begin with a $ dollar sign – for example, **$name**.
- Names can comprise letters, numbers and underscore characters, but not spaces – for example, **$subtotal_1**.
- The first character after the $ dollar sign must be a letter or an underscore character – it cannot be a number.

Note that variable names in PHP are case-sensitive, so **$name**, **$Name** and **$NAME** are three separate individual variables.

In PHP **$this** is a special variable so you cannot use “this” as a variable name.

PHP variables are “loosely typed” meaning they can contain data of any type, unlike “strongly typed” variables in some languages, where the data type must be specified when the variable is created. So a PHP variable may happily contain an integer number, or a floating-point number, or a string of text characters, or a Boolean value of **TRUE** or **FALSE**, or an object, or a **NULL** empty value.

A variable is created in a PHP script simply by stating its name. The variable can then be assigned an initial value (initialized) by using the = assignment operator to state its value. This statement, and all others in PHP, must end with a semi-colon, like this:

$body_temp = 98.6 ;

Variables that are accessible throughout the script must have unique names. See here for more on "variable scope".

The value contained within the variable can then be displayed by referencing it using the variable name, like this:

echo $body_temp ;

Usefully, the variable's value can be displayed as part of a mixed string by enclosing the string and variable name in double quotes:

echo "Body temperature is $body_temp degrees Fahrenheit" ;

The double quotes ensure that PHP will evaluate the whole string and substitute named variables with their stored values. This feature does not work if the string is enclosed in single quotes!

Do not confuse the purpose of double and single quotes. Remember that PHP only makes variable substitutions for mixed strings enclosed within double quotes.

1. Create a valid HTML document, like the one listed here, then insert PHP tags into the body section

```
<?php
  # Statements to be inserted here.
?>
```

variable.php

2. Now, insert between the PHP tags a statement to create and initialize a variable

```
$body_temp = 98.6 ;
```

3 Next, insert a statement to display the variable value alone

```
echo $body_temp ;
```

4 Then, insert a statement to display the variable value substituted in a mixed string – assigned in double quotes

```
echo "<p>Body temperature is $body_temp
                    degrees Fahrenheit " ;
```

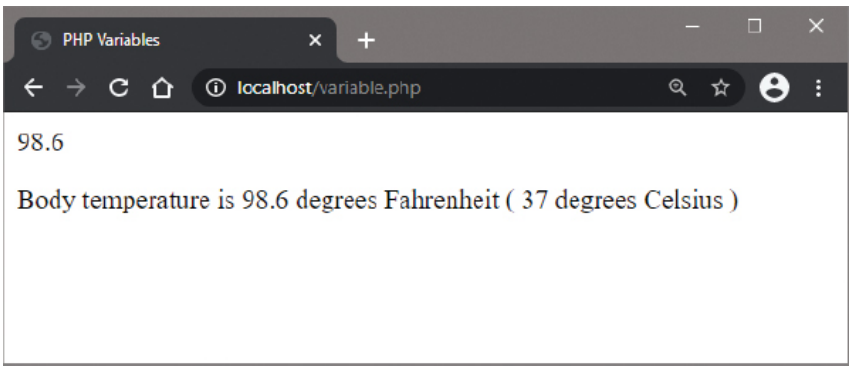
5 Insert a statement to assign a new value to the variable

```
$body_temp = 37.0 ;
```

6 Finally, insert a statement to display the new variable value substituted in a mixed string

```
echo "( $body_temp degrees Celsius )</p>" ;
```

7 Save the document in your web server's **/htdocs** directory as **variable.php** then open the page via HTTP to see the variable values get displayed



Notice that variables created in the main body of a script, like the one in this example, are accessible "globally" – throughout the entire PHP script.

Each statement in the PHP language must be terminated by a semi-colon – just as each statement in the English language must be terminated by a period.

Quoting strings

A “string” of text can be stored in a variable in much the same way as a numeric value, but the assignment must surround the string with quote marks to denote its beginning and end. Both single and double quote marks can be used for this purpose, but you must use the same type of quote marks to denote the beginning and end of the text string. For example, both these statements make valid string assignments:

$song_title = “Summertime Blues” ;

$song_title = ‘Summertime Blues’ ;

Where you wish to store a string of text that itself includes quote marks, you can “escape” the included quote marks by preceding them with a \ backslash character, or use the alternative type of quote mark within the string. For example, both these statements assign strings that include quote marks:

$song_title = “the \”Summertime\” aria by George Gershwin” ;

$song_title = ‘the “Summertime” aria by George Gershwin’ ;

The second technique, using double quote marks within the string, is easier to read and is preferred throughout this book.

Variable names cannot contain spaces – but the underscore character is often used in their place.

String values can be displayed as part of a mixed string by enclosing the mixed string in double quotes, like this:

echo “Many regard $song_title as a popular classic” ;

The double quotes ensure that PHP will evaluate the mixed string and substitute named variables with their stored values. This feature does not work if the string is enclosed in single quotes!

String values can be joined together (“concatenated”) into a single string using the **.** concatenation operator, like this:

```
$hi = ‘Hello’ ;
$bye = ‘Goodbye’ ;
$song_title = $hi . $bye ;          # ‘HelloGoodbye’
```

Additionally, spaces and punctuation can usefully be inserted when concatenating strings, so the title assignment above could be modified to include a comma and a space:

```
$song_title = $hi . ‘, ‘ . $bye ;      # ‘Hello, Goodbye’
```

1. Create a valid HTML document, like the one listed here, then insert PHP tags into the body section

```
<?php
  # Statements to be inserted here.
?>
```

string.php

2. Now, insert between the PHP tags a statement to create and initialize two variables

```
$phrase = ‘The truth is rarely pure’ ;
$author = ‘Oscar Wilde’ ;
```

3. Next, insert a statement to display a variable value alone

```
echo $phrase ;
```

4. Then, insert a statement to display the variable value substituted in a mixed string – assigned in double quotes

```
echo “<p>It is often said that <q>$phrase</q> </p>” ;
```

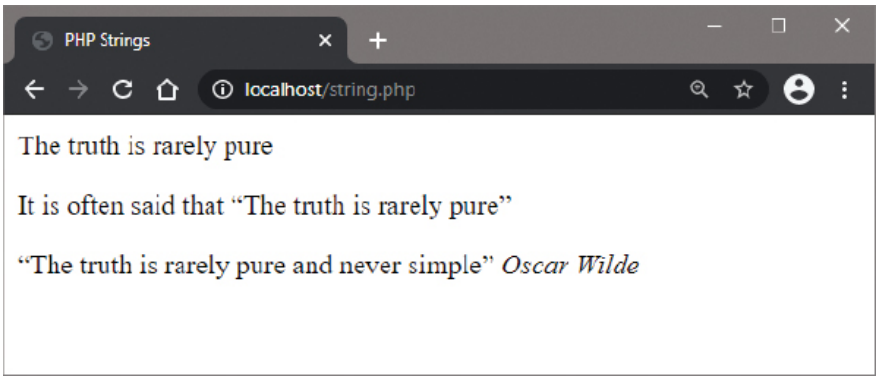
5 Insert a statement to concatenate a string to a variable

```
$phrase = $phrase . ' and never simple' ;
```

6 Finally, insert a statement to display both current variable values substituted in a mixed string

```
echo "<p><q>$phrase</q><cite>$author</cite></p>" ;
```

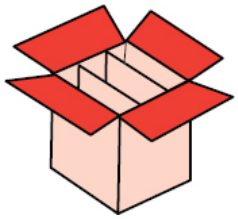
7 Save the document in your web server's **/htdocs** directory as **string.php** then open the page via HTTP to see the variable values get displayed



You can also use a shorthand concatenate assignment operator **.=** to join two strings – so that **$a = $a . $b** can be simply written as **$a .= $b**.

Adopt a consistent naming style for your variables – such as all lowercase **$my_var**, or "camelcase" **$myVar**.

Producing arrays



Indexed arrays

An array variable can store multiple items of data in sequential array “elements” that are numbered (“indexed”) starting at zero. So, the first array value is stored in array element number zero. Individual array element values can be referenced using the variable name followed by the index number in square brackets. For example, **$days[0]** references the value stored in the first array element of an array variable named “days”. Similarly, **$days[1]** references the value stored in the second array element, and so on. Variable arrays can be created by making multiple statements that assign values to successive elements, or by making a single statement using a PHP **array()** function to fill the elements:

$days[] = ‘Monday’ ; $days[] = ‘Tuesday’ ; $days[] = ‘Wednesday’ ;

or…

$days = array(‘Monday’ , ‘Tuesday’ , ‘Wednesday’) ;

Both create an array where the value stored in each element can be referenced using its index number, such as **$days[0]**.

Square brackets denote “element” so are not needed after the variable name in the assignment using the **array()** function – they are only needed when addressing an element.

Associative arrays

When creating an array, you can optionally specify a key name for each element that can be used to reference that element’s value:

$months[‘jan’] = ‘January’ ;

$months[‘feb’] = ‘February’ ;

$months[‘mar’] = ‘March’ ;

or…

$months =

array(‘jan’ => ‘January’ , ‘feb’ => ‘February’ , ‘mar’ => ‘March’) ;

Both create an array where the value stored in each element can be referenced using its key name, such as **$months[‘jan’]**.

Traversing arrays

All elements of an array can be traversed using a **foreach** loop to reference each element’s value in turn and assign it to a variable:

```
foreach( $months as $value )
{
  # Use the current $value on each iteration.
}
```

The keys can also be accessed on each iteration of the loop using

```
foreach( $months as $key => $value )
{
  # Use the current $key and $value on each iteration.
}
```

The **foreach** construct exists specifically for arrays and provides an easy way to traverse all array element keys and values. See here for more on loops.

1. Create a valid HTML document, like the one listed here, then insert PHP tags into the body section

```
<?php
  # Statements to be inserted here.
?>
```

array.php

2 Now, insert between the PHP tags a statement to create and initialize a variable array with values

```
$days = array( ‘Monday’ , ‘Tuesday’ , ‘Wednesday’ ) ;
```

3 Next, insert a statement to display the value in all array elements as a bulleted list

```
foreach( $days as $value ) { echo “&bull; $value “ ; }
```

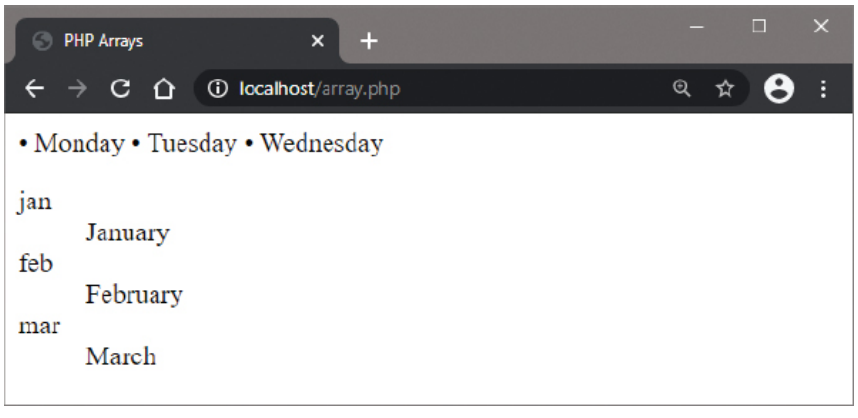
4 Then, insert a statement to create and initialize a variable array with both keys and values

```
$months = array( ‘jan’ => ‘January’ ,
                 ‘feb’ => ‘February’ , ‘mar’ => ‘March’ ) ;
```

5 Finally, insert statements to display the key and value in all elements as a definition list

```
echo ‘<dl>’ ;
foreach( $months as $key => $value )
{ echo “<dt>$key<dd>$value” ; }
echo ‘</dl>’ ;
```

6 Save the document in your web server’s **/htdocs** directory as **array.php** then open the page via HTTP to see the variable array element values get displayed



You can set the first element key to 1, so the index will begin at 1 rather than 0, using **$months = array(1 => 'January' , 'February' , March') ;**

If you specify key names you cannot reference elements by their index number, and if you specify the same key name twice, the second value will overwrite the first value.

You can easily create arrays containing sequential numbers or letters with the PHP **range()** function – for example, with

$six = range(1 , 6) ;

$a_z = range('a','z') ;

Sorting arrays

PHP provides two handy functions to convert between variable arrays and strings because they are so frequently used together. The **implode()** function converts an array to a string and inserts a specified separator between each value. Typically, this might be used to create a list of comma-separated values (csv), like this:

```
$csv_list = implode( ' , ' , $array ) ;
```

Conversely, the **explode()** function converts a string to an array by specifying a separator around which to break up the string. This might be used to create an array from a comma-separated list:

```
$array = explode( ' , ' . $csv_list ) ;
```

PHP also provides three useful functions to sort array elements into ascending alphanumeric order (a-z 1-9):

- **sort()** function – sorts by value discarding the original key.
- **asort()** function – sorts by value retaining the original key.
- **ksort()** function – sorts by key.

Existing array values are sorted with a simple statement, like this:

```
$makers = array( 'Ford' , 'Chevrolet' , 'Dodge' ) ;
sort( $makers ) ;
```

Array elements can also be sorted into descending alphanumeric order (9-1 z-a) with three similar functions:

- **rsort()** function – sorts by value discarding the original key.
- **arsort()** function – sorts by value retaining the original key.
- **krsort()** function – sorts by key.

It is important to recognize that specified key names will be discarded by the **sort()** and **rsort()** functions, so these should only be used where the key/value relationship is not significant – otherwise use the **asort()** and **arsort()** functions to retain the keys.

You can test if a variable is an array using the **is_array()** function – for example, **is_array($var) ;**

1. Create a valid HTML document, like the one listed here, then insert PHP tags into the body section

```
<?php
  # Statements to be inserted here.
?>
```

sort.php

2. Now, insert between the PHP tags a statement to create and initialize a variable array with key names and values

```
$cars = array( 'Dodge' => 'Viper' ,
        'Chevrolet' => 'Camaro' , 'Ford' => 'Mustang' ) ;
```

3. Next, insert statements to display all keys and values of the array as a bulleted list within a definition list

```
echo '<dl><dt>Original Element Order :<dd>' ;
foreach( $cars as $key => $value )
{ echo ' &bull; ' , $key . ' ' . $value ; }
```

4. Then, insert statements to display the array sorted by value

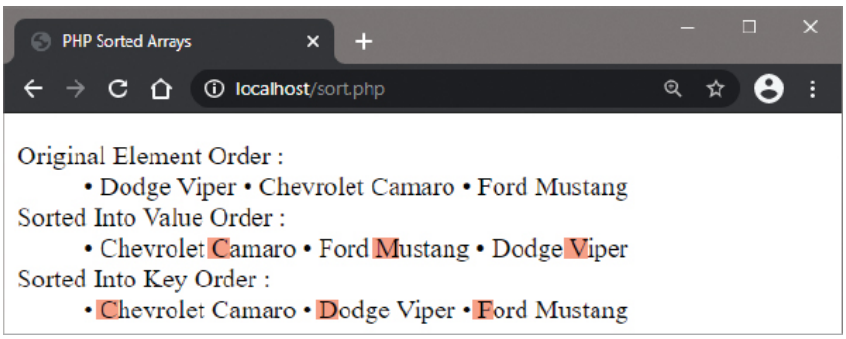
```
asort( $cars ) ;
echo '<dt>Sorted Into Value Order :<dd>' ;
foreach( $cars as $key => $value )
```

```
{ echo ' &bull; ' , $key . ' ' . $value ; }
```

5 Finally, insert statements to display the array sorted by key

```
ksort( $cars ) ;
echo '<dt>Sorted Into Key Order :<dd>' ;
foreach( $cars as $key => $value )
{ echo ' &bull; ' , $key . ' ' . $value ; }
echo '</dl>' ;
```

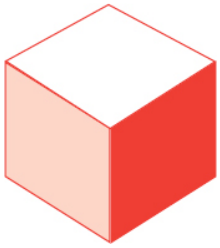
6 Save the document in your web server's **/htdocs** directory as **sort.php** then open the page via HTTP to see the sorted array values get displayed



You can discover the number of elements in an array with a **count()** function – for example, **$num = count($array) ;**

Do not use the **sort()** or **rsort()** functions where the keys are important.

Describing dimensions



The numbering of array elements is more correctly known as the “array index” and, because element numbering starts at zero, is sometimes referred to as a “zero-based index”. An array created with a single index is a one-dimensional array in which the elements appear in a single row:

Element content	A	B	C	D	E
Index numbers	[0]	[1]	[2]	[3]	[4]

Arrays can also have multiple indices – to represent multiple dimensions. An array created with two indices is a two-dimensional array in which the elements appear in multiple rows:

First index	[0]	A	B	C	D	E
	[1]	F	G	H	I	J
Second index		[0]	[1]	[2]	[3]	[4]

With multi-dimensional arrays, the value contained in each element is referenced by stating the number of each index. For example, with the two-dimensional array above, the element at [1][2] contains the letter H.

Two-dimensional arrays are useful to store grid-based information, such as XY coordinates. Creating a three-dimensional array, with three indices, would allow XYZ coordinates to be stored.

Creating a multi-dimensional array in PHP is simply a matter of creating an outer array, whose elements are themselves arrays. As usual, you may optionally specify a key name for each element – in this case to represent each inner array. Individual values can then be referenced using the key name and inner index number:

$matrix[‘Letter’][0]

It is, however, necessary to always enclose array variables that use quoted keys within curly braces when included in a string:

echo “Element value is {$matrix[‘Letter’][0]} ” ;

All inner arrays in a multi-dimensional array can be accessed using a **foreach** loop to iterate through the arrays. A nested **foreach** loop can then reference each element in turn.

Multi-dimensional arrays with more than two indices can produce hard to read source code and may lead to errors.

1. Create a valid HTML document, like the one listed here, then insert PHP tags into the body section

```
<?php
  # Statements to be inserted here.
?>
```

matrix.php

2. Now, insert between the PHP tags a statement to create and initialize two regular variable arrays and a two-dimensional array with key names

```
$letters = array( ‘A’ , ‘B’ , ‘C’ ) ;
$numbers = array( 1 , 2 , 3 ) ;
$matrix =
array( ‘Letter’ => $letters , ‘Number’ => $numbers ) ;
```

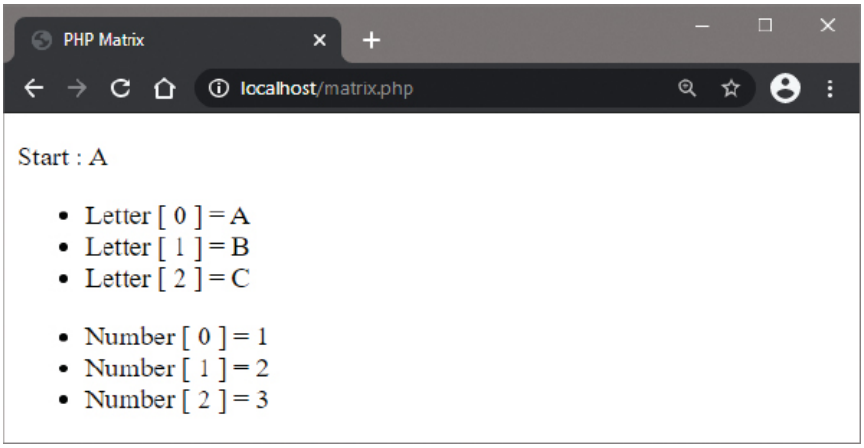
3. Next, insert a statement to display a single stored value

```
echo “<p>Start : {$matrix[‘Letter’][0]} </p>” ;
```

4. Finally, insert statements to display the key and value in all elements as two unordered lists

```
foreach( $matrix as $array => $list )
{
  echo '<ul>' ;
  foreach( $list as $key => $value )
  { echo "<li>$array [ $key ] = $value " ; }
  echo '</ul>' ;
}
```

5 Save the document in your web server's **/htdocs** directory as **matrix.php** then open the page via HTTP to see the variable array values get displayed



Multi-dimensional arrays are more common than you might think. For example, an HTML form that allows multiple selections for inputs with the same name that are submitted as a multi-dimensional array.

Array variables that use quoted keys must be enclosed within curly braces when included in a string.

Checking types

PHP variables can store various types of data so it is useful to clearly understand each data type in detail:

- **string** – a series of characters in which each character is the size of one byte, up to a maximum length of 2GB. Strings enclosed in single quotes are treated as literals, whereas strings in double quotes will interpret special characters.
- **int** – a non-decimal number between 2,147,483,648 and -2,147,483,647, specified as a decimal (10-based), hexadecimal (16-based – prefixed with 0x), or octal (8-based – prefixed with 0).
- **float** – a floating-point decimal number or a number in exponential format. A float is also known as a "double".
- **bool** – an expression of a Boolean truth value, which may be only true or false.
- **array** – an ordered map of multiple data values that associates keys to values. The keys are index numbered by default or may be explicitly-specified labels.
- **object** – a class containing stored data properties and providing methods to process data.
- **resource** – a reference to an external resource that is created and used by special functions.
- **NULL** – a variable with no value whatsoever. It may have not been initialized, have been assigned the **NULL** constant, or have been nullified by the PHP **unset()** function.

The data type of any item can be checked by specifying it as the argument to the **gettype()** function. For example, where a variable **$num** contains an integer value, **gettype($num)** will return "integer", and where a variable **$str** contains a string value, **gettype($str)** will return "string", and so on. Where the data type cannot be identified, the **gettype()** function will return "unknown type".

Perhaps more usefully, the **var_dump()** function can check an item's data type and will dump structured information that displays its type and value.

The creation and use of PHP class objects are demonstrated fully in Chapter 7.

1. Create a valid HTML document, like the one listed here, then insert PHP tags into the body section

```
<?php
  # Statements to be inserted here.
?>
```

types.php

2. Next, insert between the PHP tags a statement to create a filestream resource and an array

```
$filestream = fopen( 'index.html' , 'r' ) ;
$data = array( 'PHP' , 1 , 2.3 , TRUE , NULL , array() ,
                              new Directory , $filestream ) ;
```

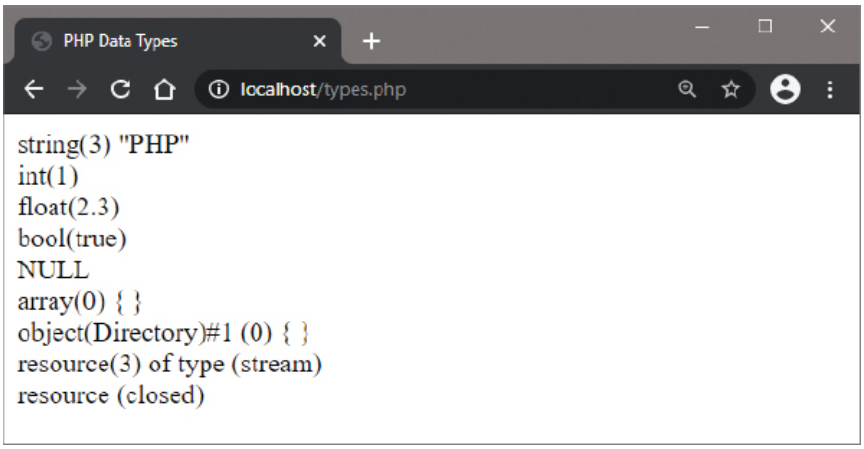
3. Now, insert a statement to display the data type stored within each array element

```
foreach( $data as $type )
{
  var_dump( $type ) ;
  echo '<br> ' ;
}
```

4. Finally, insert statements to destroy the filestream resource then attempt to get its data type once more

```
fclose( $filestream ) ;
echo gettype( $filestream ) ;
```

5. Save the document in your web server's **/htdocs** directory as **types.php** then open the page via HTTP to see the data types and values get displayed



The creation and use of PHP filestream resources are demonstrated fully in Chapter 8.

The Boolean constants **TRUE** and **FALSE** are not case-sensitive, so may be written as **true** and **false**.

Defining constants

Fixed data values that will never change in a script should be stored in a PHP “constant”, rather than in a variable. A constant is a named container in which numeric or string values can be stored and can be referenced using the constant’s name.

Unlike a variable, a constant cannot contain Boolean values of **TRUE** or **FALSE**, nor represent an object, nor a **NULL** empty value. Additionally, a constant cannot be changed as the script proceeds. This safeguards the constant value from accidental alteration and is good programming practice.

Constant names are subject to the same naming conventions as variables, but it is traditional to use uppercase characters for constant names – to easily distinguish them from variable names. Constants are created using a **define()** function to specify the constant’s name and the value it will contain, like this:

define(*‘NAME’* , *value*) ;

Most significantly, constant names are not prefixed by a **$** dollar sign, so they cannot be included in mixed strings for display – as PHP cannot distinguish a constant name from a regular capitalized string. Consider these statements, for example:

```
define( ‘USER’ , ‘Mike’ ) ;
echo “Hello USER”
```

…the output here is **Hello USER**, not **Hello Mike** as desired.

In order to include a constant’s value in an output string it must be referenced separately and “concatenated” to the string with the PHP . concatenation operator, like this:

```
echo ‘Hello ‘ . USER ;
```

…the output here is now **Hello Mike** as desired.

Array constants, in which all elements contain fixed data values, can also be created using the **define()** function, like this:

define('WEEKDAYS' , ['Mon', 'Tue' , 'Wed' , 'Thu' , 'Fri']) ;

Constants defined by the script author can be used just like the predefined constants that are built into PHP, such as **PHP_VERSION** and **PHP_OS** that describe the version and server operating system.

When creating a variable, always consider whether its value will ever change – if it will not, you should create a constant instead; for example, **define('PI' , 3.14) ;**

1. Create a valid HTML document, like the one listed here, then insert PHP tags into the body section

```
<?php
  # Statements to be inserted here.
?>
```

constant.php

2. Now, insert between the PHP tags a statement to create and initialize a constant

```
define( 'USER' , 'Mike' ) ;
```

3. Add a statement to create and initialize an array constant

```
define( 'PETS' , [ 'Kitten' , 'Puppy' , 'Hamster' ] ) ;
```

4. Next, insert a statement to display two constant values in a concatenated string

```
echo '<p>Hello ' . USER . ' how is your ' . PETS[1] . '?</p>' ;
```

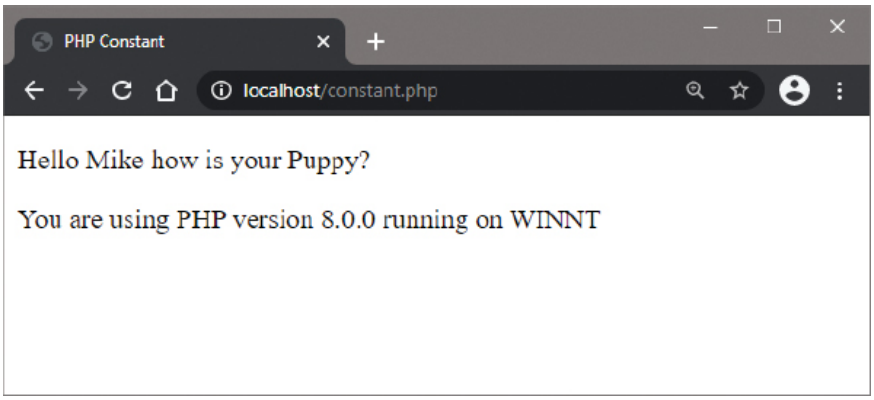
5. Insert a statement to display the predefined constant value of the PHP version in a concatenated string

```
echo '<p>You are using PHP version ' . PHP_VERSION ;
```

6 Finally, insert a statement to display the predefined constant value of the host operating system in a string

```
echo ‘ running on ‘ . PHP_OS . ‘</p>’ ;
```

7 Save the document in your web server’s **/htdocs** directory as **constant.php** then open the page via HTTP to see the results get displayed



You cannot include constants in mixed strings, or change their value, or begin their name with a dollar sign.

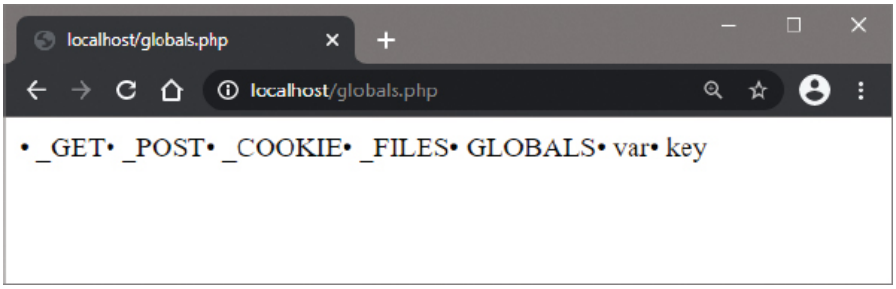
You should always use uppercase for constant names to easily differentiate them from variables in your code.

Exploring superglobals

PHP provides several predefined variables that are always globally available anywhere within any script. These are aptly known as “superglobals” and are described briefly here:

- **$GLOBALS** – an associative array containing references to all variables that are currently defined in the global scope of the script, according to context. The variable names are the keys of this associative array – for example:

  foreach($GLOBALS as $key => $var) echo “• $key “;



- **$_SERVER** – an array containing information such as headers, paths and script locations that is arbitrarily provided by the web server.
- **$_GET** – an associative array of variables passed to the script by appended URL parameters.
- **$_POST** – an associative array of variables passed to the script by the HTTP POST method.
- **$_FILES** – an associative array of uploaded file items passed to the script by the HTTP POST method.
- **$_COOKIE** – an associative array of variables passed to the script via HTTP from cookies on the user’s computer.
- **$_SESSION** – an associative array containing variables available to each script on a website until the user exits.
- **$_REQUEST** – an associative array that by default contains the contents of **$_GET**, **$_POST** and **$_COOKIE**.
- **$_ENV** – an associative array of variables passed to the current script by the shell environment under which the PHP parser is running. Different systems will run different kinds of shells.

The **$_FILES** superglobal is used by the file upload example here.

The **$_POST** superglobal is used by the HTML form submission example here.

The **$_COOKIE** superglobal is used by the example here to get data from a cookie on the user's computer.

The **$_SESSION** superglobal is used by the example here to store data on the web server.

1. Create a valid HTML document, like the one listed here, then insert PHP tags into the body section

```
<?php
  # Statements to be inserted here.
?>
```

supers.php

2. Now, insert between the PHP tags a statement to display your server software details

```
echo 'Web Server : '.$_SERVER[ 'SERVER_SOFTWARE' ].'<br>' ;
```

3. Add a statement to display this script name

```
echo 'This Script : '.$_SERVER[ 'PHP_SELF' ].'<br>' ;
```

4. Next, insert statements to display the host name and the page request method

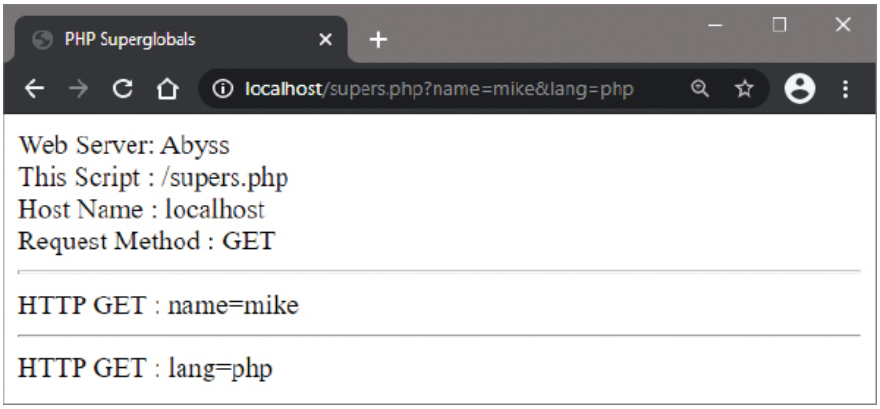
```
echo 'Host Name : '.$_SERVER[ 'HTTP_HOST' ].'<br>' ;
echo 'Request Method : '.$_SERVER[ 'REQUEST_METHOD' ] ;
```

5. Insert a statement to display appended URL parameters

```
foreach( $_GET as $key => $value )
{ echo '<hr>HTTP GET : '.$key.'='.$value ; }
```

6. Save the document in your web server's **/htdocs** directory as **supers.php**

7. Open the page via HTTP by appending parameters to the script name in the browser address field, such as **?name=mike&lang=PHP**, to see them pass to the script



$_SERVER['PHP_SELF'] is used by the example here to create a "sticky" HTML form.

$_SERVER['REQUEST_ METHOD'] is used by the example here to establish whether an HTML form has been submitted by a user.

The HTTP **GET** method is used by the example here to set parameters for an XML Web Service response.

Summary

- A variable is a named container in which changeable data can be stored.
- The value stored in a variable can be referenced using that variable's name.
- Variable names must begin with a **$** dollar sign, may comprise letters, numbers and underscore characters, and the first character after the **$** must only be a letter or an underscore.
- A variable can store an integer number, floating-point number, text string, Boolean value, object, or **NULL** value.
- A variable is initialized using the **=** assignment operator to assign an initial value to that variable.
- A variable's value can be displayed as part of a mixed string by enclosing the string and variable name in double quotes only.
- Strings may be enclosed in either single quotes or double quotes, and inner quote marks can be escaped using a backslash.
- String values can be joined together into a single string using the **.** concatenation operator.
- An array variable can store multiple items of data in sequential array elements that are numbered starting at zero by default.
- Optionally, key names may be specified for each array element.
- A **foreach** loop can traverse each element in an array.
- The **implode()** and **explode()** functions convert between strings and arrays by adding or removing specified separators.
- Arrays can be sorted into alphanumeric order by value or key.
- A multi-dimensional array is simply an array whose element values are themselves arrays.
- The **gettype()** and **var_dump()** functions can be used to check data types.
- A constant is a named container in which a fixed integer, floating-point number, or text string can be stored.
- Constant names are not prefixed by a **$** and are all uppercase.
- A constant is initialized using the **define()** function.

- Superglobal variables are always globally available anywhere within any script.